

PreDoc

Full conversation summary: from problem statement to hackathon build & debugging log |
Anchor track: Gen AI | Rotating track: Health Tech

1. The Original Problem Statement

Patients in overcrowded hospitals, particularly government and public hospitals, face delays and incomplete care because their medical history and previous records are fragmented and difficult to access. Doctors have limited consultation time to collect detailed histories and review paper prescriptions, reports, and discharge summaries, while patients requiring urgent or specialised care may also face delays in being directed to the appropriate facility. This results in repeated questioning, incomplete clinical information, inefficient consultations, and critical time lost.

2. How the Idea Evolved

The starting idea was a pre-consultation AI copilot that builds a doctor-ready case file before the patient sees the doctor, instead of just another symptom-checker chatbot. Early research grounding included a 2024 systematic review of AI history-taking chatbots (found benefits in engagement and data collection, but flagged unresolved data-protection questions and a need to test with non-tech-savvy patients) and India's ABDM/ABHA infrastructure, which already standardises records as FHIR resources with consent-based sharing.

A key check along the way was whether this idea was genuinely new or just re-packaging existing products. Prior art considered:

- **Ambient scribes** (Abridge, Suki, EkaScribe/Eka Care) — document the consultation itself, in real time, while the doctor talks to the patient.
- **ABDM/ABHA** — solves record interoperability once a digital record already exists.
- **DASI (Göttingen study)** — a validated German research app for pre-consult intake, but tablet/checkbox-based, assumes literacy and device access, and doesn't touch old paper records, OCR, or triage/routing.

Conclusion: the pre-consult intake concept is not new, but the specific combination the team is building — voice-first vernacular intake, OCR of physical paper records, real-time triage flagging, and kiosk/IVR delivery for non-smartphone, low-literacy patients in public hospitals — fills a genuine gap none of the above cover.

Recommended answer if a judge says "this already exists"

Agree with the premise, then narrow to the specific gap: "You're right that pre-consult AI intake exists — there's a German study (DASI) on exactly that, and ambient scribes like Abridge and EkaScribe already turn consultations into notes. What none of them handle is [X] — which is the actual bottleneck in a government hospital OPD, not a private clinic."

If judge says...	Response
"Ambient scribes already do this"	Those transcribe the consultation while it happens. PreDoc works before that, so the consult itself starts with a ready case file.
"There's an app for this already (DASI-type)"	DASI needs a literate patient tapping a tablet UI and doesn't touch old paper records at all. PreDoc is voice-first, vernacular, and OCRs old paper prescriptions.

If judge says...	Response
"ABDM already solves this"	ABDM only helps if a digital record already exists. Most public-OPD patients have no ABHA record — only paper. ABDM is an optional add-on, not the foundation.
"Triage systems already exist"	Most triage is a nurse asking a few questions at a counter. PreDoc's flagging happens as a byproduct of intake already in progress, at zero extra time cost.

Research grounding: the DASI paper (Göttingen, Germany)

DASI is an iPad app for symptom-oriented, checkbox-based history taking in general practice, validated against face-to-face interviews. It confirms the underlying problem (7.6-minute average GP consults in Germany, unstandardised paper questionnaires, tedious manual transcription into records) but also flags an open risk: it is not yet proven that patient self-report through an app matches what a trained interviewer would elicit, especially for red-flag symptoms. Only about 5% of German GPs used any digital history tool as of 2020, despite nearly 30% wanting one — showing adoption fails on trust and friction, not on demand.

This directly justifies PreDoc's core safety design: source-linked facts (traceable to transcript/photo) and mandatory doctor sign-off before anything is saved to the record.

3. Naming

The project was originally named **Sahayak**, later renamed to **PreDoc** — short, self-explanatory ("before the doctor"), and easy to say in a live pitch.

4. Final Solution: Feature-by-Feature

Feature	Description
Voice-First, Vernacular Intake	Patient speaks their history in their own regional language, guided through chief complaint, symptoms, past history, medications, and allergies.
Dual-Mode Adaptive Interface	Speak naturally or tap large icons; questions adapt to the complaint. Built for low-literacy, non-smartphone users.
Smart Document Chronology Timeline	Old prescriptions and reports are OCR-scanned and plotted as a chronological treatment/lab timeline for the doctor.
Source-Linked Case Draft	Transcript and timeline merge into a structured SOAP-style draft, with every fact traceable back to its source.
Instant Red-Flag Interceptor	Flags time-critical symptom patterns in real time and alerts triage staff, ahead of the queue.
Doctor Review & Approval	Doctor reviews the draft beside original sources and approves in under a minute; nothing saves without sign-off.

Note: a fourth idea — a separate AYUSH (Ayurveda) clinical-questioning engine alongside the standard SOAP/allopathy path — was deliberately left out of the MVP. It's a strong, India-specific differentiator but needs deep domain accuracy to avoid looking superficial; kept as a roadmap item rather than something built in 36 hours.

5. End-to-End Workflow

#	Step	Description
1	Voice-first, vernacular intake	Patient speaks their history in their own language; system guides through chief complaint -> duration/symptoms -> past history -> medications -> allergies.
2	Dual-mode adaptive interface	Patient can switch to tapping large icons instead of speaking, at any point, for low-literacy or privacy reasons; feeds the same data structure.
3	Smart document chronology timeline	In parallel, old prescriptions/reports are photographed, OCR-scanned, and plotted as a chronological timeline of treatments and labs.
4	Source-linked case draft	Transcript + document timeline merge into a structured SOAP-style draft; every fact is traceable back to its transcript line or document source.
5	Instant red-flag interceptor	While the draft is built, symptom patterns tied to time-critical conditions are checked continuously; a match immediately flags urgency and alerts triage.
6	Doctor review & approval	Doctor reviews the draft next to original sources, edits if needed, and approves; nothing is saved to the record until sign-off.

6. Tech Stack

Layer	Choice
Frontend	React + Tailwind CSS
Backend	FastAPI (Python) or Express.js (Node.js)
Database	PostgreSQL or Firebase Firestore
AI/ML	Google Gemini API only (multimodal: voice transcription, OCR extraction from photographed prescriptions/reports, and structured case-file generation) — chosen over Claude/GPT for cost, since Gemini has a generous free tier, ideal for a hackathon build
Visualization	Recharts / Chart.js for the document timeline
Deployment	Vercel/Netlify (frontend), Render/Railway (backend)

Reasoning for Gemini as the single AI engine: it natively accepts text, image, and audio in one API, which maps directly onto the OCR (photographed prescriptions) and voice (patient conversation) needs without stitching together separate speech-to-text and vision services — simplifying the architecture for a 36-hour build.

7. 36-Hour Build Plan

Time	Task	Owner
Hours 0-4	Finalise scope, wireframe 3 screens (patient intake, doctor dashboard, nurse alert), set up repo, API keys, base scaffolding.	Whole team
Hours 4-14	Build core intake pipeline: dual-mode UI (voice + touch), STT integration, guided conversation script, save transcript to backend.	Frontend + 1 backend dev
Hours 14-22	Build document pipeline: photo upload, vision-LLM extraction, unit normalisation, timeline chart on doctor dashboard.	1-2 backend/AI devs
Hours 22-28	Build red-flag rules engine and alert UI; wire into the live intake flow so flags appear in real time.	1 backend dev
Hours 28-32	Integrate end-to-end: check-in -> intake -> case draft with timeline -> doctor review -> flag/routing.	Whole team
Hours 32-36	Testing, bug fixes, polish doctor dashboard visuals, rehearse live demo + record backup video.	Whole team

8. Step-by-Step Build Prompts (for Antigravity IDE)

Use Planning mode for every prompt below (multi-file tasks). Verify each step works in the live preview before moving to the next one. Load API keys from a .env file; never hardcode secrets.

Prompt 1 — Project scaffold

Create a full-stack web app called PreDoc with this structure:

- Frontend: React + Tailwind CSS (Vite setup)
- Backend: FastAPI (Python)
- Database: PostgreSQL, with SQLAlchemy models

Set up three tables: patients (id, name, age, language), visits (id, patient_id, status, urgency_flag, department), and case_drafts (id, visit_id, content_json, source_links_json). Create a basic health-check API route and confirm the frontend can call it and display "PreDoc backend connected."

Do not add authentication yet. Load all secrets from a .env file and add .env to .gitignore.

Prompt 2 — Voice-first, vernacular intake

Add a patient intake flow. Create a backend endpoint /api/intake/start that begins a guided conversation using the Gemini API, following this fixed clinical script in order: chief complaint, duration, associated symptoms, past history, medications, allergies. The frontend should have a microphone button that records patient speech, sends the audio to the backend, and the backend uses Gemini to transcribe and continue the guided conversation, returning the next question each time. Store each transcript turn against the visit_id. Support at least 2 languages (English and Hindi) – detect language from the response or let the patient pick it at the start.

Prompt 3 — Dual-mode adaptive interface

Add a second intake mode to the same intake screen: a touch/icon mode. Show a toggle at the top ("Speak" / "Tap") that lets the patient switch anytime. In tap mode, show large icon buttons for common symptoms (fever, pain, cough, breathlessness, injury, other) and follow-up yes/no or multiple-choice buttons instead of voice. Both modes must write to the same visit transcript data structure used in the voice mode, so the backend doesn't need to know which mode was used.

Prompt 4 — Smart document chronology timeline (confidence-aware)

Update the document upload feature for prescription/report extraction with a stricter, safer pipeline. Instruct Gemini (vision input) to only report fields it can read with high confidence, marking illegible/ambiguous handwriting fields as "unclear" instead of guessing a plausible drug name. Extract into JSON with a confidence level (high/medium/low) per medication/diagnosis field, plus raw_text as actually seen. Normalize units (mg/g, mmHg). Store results and the original image against the visit_id. On the frontend, show a green "Verified" badge only for high confidence and a yellow "Low confidence – please verify" badge otherwise, with the original photo viewable alongside, a manual correction option, and a Recharts timeline of extracted dates. No field should reach the final case draft unless high-confidence or manually verified.

Prompt 5 — Source-linked case draft (accurate, case-specific)

Create a backend endpoint /api/visits/{id}/generate-draft that merges the intake transcript and extracted document data into a structured SOAP-style case draft as JSON. Subjective must pull actual reported symptoms/duration/severity verbatim, not a vague paraphrase. Objective only includes data that actually exists, explicitly stating "none uploaded" when true – never fabricated. Assessment is based strictly on what was reported (specific symptoms, duration, associated factors, red flags), with no diagnosis suggested. Plan gives process-level next steps specific to the complaint, not generic boilerplate. If information is insufficient for a field, say so explicitly. Every field includes a source reference to the exact transcript turn or document it came from. Display on a /case/{visit_id} page where clicking a fact highlights its source.

Prompt 6 — Instant red-flag interceptor

Add a real-time red-flag checker that runs during intake (after each transcript turn is saved). Create a rules list of symptom-combination triggers, for example: "chest pain" + "breathlessness" -> urgent, cardiac; "sudden severe headache" + "vision changes" -> urgent, neuro. When a match is found, immediately set the visit's urgency_flag and department field, and show a red banner on the frontend: "Urgent: recommend immediate attention, suggested department: {department}." Also send this flag as a notification to a separate /triage dashboard page listing all urgent visits in real time.

Prompt 7 — Doctor review & approval

Create a doctor dashboard page /doctor listing all visits with a pending case draft. Clicking a visit opens a review screen showing the generated case draft next to the original sources (transcript and uploaded document images side by side). Add Edit and Approve buttons. Editing lets the doctor modify any field inline. Approve sets the visit status to "approved" and locks the draft as final – nothing should be treated as final until this happens.

Prompt 8 — Demo polish & seed data

Add a script to seed the database with 3 realistic demo patients: one routine case, one with an old prescription to upload, and one that should trigger the red-flag urgent path. Add a simple landing page for the kiosk view (/kiosk) that starts a new visit and routes to intake. Test the full flow end-to-end for all 3 seeded patients and fix any errors found during that walkthrough.

Tip: commit/save working code after every successful prompt so a mid-build failure doesn't cost progress.

9. Debugging Log: Antigravity "User cancelled agent execution"

This is a known, fairly common Antigravity issue reported by many users, not a mistake in the prompt. "User cancelled agent execution" together with "Failed to fetch" usually points to one of the following:

- **Network interruption mid-stream** — Antigravity streams responses continuously; a brief connection drop can be misreported as a cancellation.
- **Rate limits or quota exhaustion** — running many prompts back-to-back can hit Gemini API or Antigravity's internal quota, aborting execution without a clear quota message.
- **A hung background process** — if a terminal command (e.g. starting a dev server) hangs or crashes, the framework can misreport it as a cancellation.
- **A server-side bug** — this has been an ongoing, widely-reported issue on Google's own support forum, sometimes affecting many users at once regardless of prompt content.

Recommended fixes, in order

- Simply retry the exact same prompt — fixes it a large fraction of the time.
- Check for a stable network connection, especially on shared/hackathon Wi-Fi.
- Open the terminal panel and kill any hung process from the previous prompt before retrying.
- Switch models (e.g. Gemini Flash vs Pro) if available.
- Restart Antigravity if retries keep failing, to clear stuck internal state.

Hackathon safeguard: commit or manually back up working code after every successful prompt so this failure mode never costs lost progress.

10. Debugging Log: Prescription OCR Hallucination

During testing, a real handwritten prescription (R.K. Hospital, illegible cursive handwriting) was uploaded. The app confidently returned a clean-looking medication list (Metformin HCl 500mg, Paracetamol 1g, Amlodipine 5mg, Atorvastatin 20mg) that did not match what was actually written on the page. This was diagnosed as **hallucination, not an extraction bug** — the vision model defaulted to statistically common, plausible-sounding drug names instead of admitting it couldn't read the handwriting. This is a known failure mode for vision-LLMs on illegible input, and a serious risk for a medical tool since a confident wrong answer is more dangerous than an obvious error.

Fix prompt used

```
Update the document upload feature for prescription/report extraction with a stricter, safer pipeline. Instruct Gemini to only report fields it can read with high confidence, and to mark unclear handwriting as "unclear" per field instead of guessing a plausible value. Extract into a JSON structure with a confidence level (high/medium/low) per medication and diagnosis field, plus the raw_text as actually seen. On the frontend, show a green "Verified" badge only for high confidence, and a yellow "Low confidence - please verify" badge otherwise, with the original photo viewable alongside. Add manual correction so staff can overwrite any field, tagged as "manually verified." No field should reach the final doctor-facing case draft unless it is either high-confidence or manually verified by a human.
```

Why the feature is still kept in the MVP

- Printed/typed prescriptions and lab reports extract reliably — the handwriting problem is narrower than "OCR doesn't work."
- Even partial, flagged extraction saves doctor time versus reading the whole paper themselves.
- The chronology/timeline value degrades gracefully even with some uncertain fields.
- It is the necessary building block for the Smart Document Chronology Timeline — the single strongest "wow factor" demo feature.
- Honest, confidence-aware extraction is a stronger pitch than claiming perfect OCR, since no vision model today reliably reads illegible doctor handwriting — naming this limitation proactively lands better with judges than being caught fabricating data live on stage.

Demo guidance: use the "Load sample Rx" button with a clean, legible sample as the primary demo path; if showing the handwriting-handling feature, frame it explicitly as "here's what happens when input is unclear" (the yellow low-confidence badges and manual correction flow), not as a lucky guess.

11. Debugging Log: Voice Intake Returning "[inaudible]"

The voice intake screen returned "[inaudible]" for every patient response, even when speech was clear. Diagnosed as a likely frontend recording issue (mic permission not granted, recording cut off too early, wrong audio MIME type sent to Gemini, or an empty/corrupted audio blob) rather than a transcription failure by Gemini itself.

Fix prompt used

```
Voice intake always returns "[inaudible]" even when the user speaks clearly. Fix the audio pipeline: (1) verify mic permission is granted, show error if denied; (2) ensure MediaRecorder captures chunks correctly and recording isn't cut off too early (reject recordings under 0.5s); (3) log captured audio blob size/MIME type to console before sending, to confirm real audio is captured; (4) pass the correct MIME type (audio/webm or audio/wav) to the Gemini API call; (5) show a recording-level indicator so the user knows audio is being picked up; (6) if Gemini genuinely can't transcribe, show "Couldn't hear that, try again" instead of silently displaying "[inaudible]".
```

Debug tip given: play back the recorded audio locally first (temporary <audio controls> element) — if the patient's own voice isn't audible in playback, the bug is confirmed to be in the recording step, not Gemini.

12. Debugging Log: App Stuck on "Connecting..." / "Starting consultation..."

The app's status pill stayed stuck on "Connecting..." and the start-consultation button spun indefinitely. Initial checks ruled out the Gemini API key, CORS, and prompt design. After a full agent-led investigation (logs, direct API tests, port checks, CORS preflight tests), three stacked root causes were found:

- **SQLAlchemy stdout pipe blockage** — `echo=(settings.ENVIRONMENT == "development")` in `database.py` printed every SQL query executed during background polling, producing over 438,000 lines of logs. On Windows this exhausted the process's stdout pipe buffer and completely froze Uvicorn, so it stopped responding to any request at all.
- **Missing Gemini HTTP timeout** — the Google GenAI client had no socket/HTTP timeout, so a slow network handshake could block for minutes while cycling through candidate models.
- **Frontend polling and missing error UI** — `App.jsx` reset `loading=true` on every 6-second poll with no `AbortController` timeout, trapping the header in "Connecting..."; the start screen had no error display, leaving the button in an infinite spinner regardless of what the backend was actually doing.

Resolution

- Disabled verbose SQL query logging in development mode.
- Configured the Gemini client with an explicit HTTP timeout (10s) and fallback logic that skips stalled models instead of hanging on them.
- Configured CORS explicitly for local dev origins and verified via a preflight `OPTIONS` request (returned 200 OK with correct headers).
- Added `AbortController` timeouts on the frontend and a visible error card with a "Retry consultation" button instead of an infinite spinner.
- Made background health-check polling silent so the status pill stays steady at "Online" instead of flapping to "Connecting..." on every poll.

Verified live: status pill shows "Online," and consultation now starts instantly on click. Lesson: the bug had nothing to do with the AI feature being built — it was an unrelated logging setting silently choking the server. Worth a quick sanity check of the backend terminal before demos to confirm no SQL log spam is occurring again.

13. Debugging Log: Slow Responses (25-30+ seconds)

After the freeze was fixed, requests still took a long time. Chrome DevTools Network tab confirmed real backend latency of 25,000-30,000+ ms per request — consistent with the fallback logic trying multiple Gemini models sequentially (roughly 3 models × ~10s timeout each).

Fix prompt used

```
Gemini API calls are taking 25-30+ seconds because _generate_with_fallback tries multiple models sequentially, each with a ~10s timeout, before one succeeds. Fix: (1) reorder the candidate model list so the fastest, most reliable model (e.g. gemini-2.5-flash) is tried first; (2) reduce per-model timeout to 5 seconds; (3) log which model actually succeeded and how long it took, to identify and remove consistently failing models from the fallback list; (4) cap total fallback attempts to 2 models max; (5) add a frontend "Thinking..." indicator immediately on request start so even a 5-8 second wait doesn't feel broken.
```

14. Debugging Log: Tap-Mode "Allergies" Step Showing No Input

On the final intake step (Step 6 of 6, Allergies), the question text rendered correctly but no input controls appeared — no Yes/No buttons, no text field. Diagnosed as the tap-mode component likely expecting a fixed Yes/No or icon-list format, while this open-ended question's data (empty or unexpected options array) caused the input area to render nothing, silently.

Fix prompt used

Step 6 "Allergies" shows the question but no input controls (no Yes/No or text field) – silently blank. (1) Ensure backend sends `input_type` ("yesno" or "text") for every step, including open-ended ones like allergies. (2) Frontend: if open-ended, render a text input or multi-select chips (Penicillin, Peanuts, Dust, None, Other) instead of nothing. (3) Add fallback: default to plain text input if no options/`input_type` is present. (4) Log a console warning if a step has no options and no `input_type`, so this fails loudly in testing, not silently in the UI.

15. Debugging Log: Generic / Inaccurate SOAP Case Draft

The generated case draft (Objective, Assessment, Plan sections, each tagged with a source-linked "Transcript #ID" badge) was rendering correctly and honestly — e.g. Objective correctly stated "No prior medical documents or lab reports uploaded" rather than fabricating data. However the Assessment and Plan sections were too generic ("Requires targeted clinical examination and diagnostic baseline confirmation") rather than reflecting the specific patient case.

Fix prompt used

Update the case-draft generation prompt so: Subjective pulls actual reported symptoms, duration, and severity verbatim from the transcript (not a paraphrase that drops specifics). Objective only includes data that actually exists, with an explicit "none uploaded" statement when true – never fabricated placeholder values. Assessment is based strictly on what was reported (specific symptoms, duration, associated factors, red flags) instead of generic boilerplate, and must not suggest a diagnosis. Plan suggests only process-level next steps relevant to the specific complaint (e.g. "physician to examine [reported symptom]", "reassess if symptoms persist beyond x days"), not a fixed generic template repeated for every patient. Every field keeps its source-linked "Transcript #ID" tag. If information is genuinely insufficient for a field, say so explicitly rather than filling in vague, safe-sounding boilerplate.

16. Pre-Demo Checklist ("What's Next")

#	Task	Detail
1	Verify Red-Flag Interceptor end-to-end	Run an intake with a red-flag symptom combo (e.g. chest pain + breathlessness); confirm it flags urgency live and appears immediately on the Triage Dashboard.
2	Verify Doctor Review & Approval end-to-end	Open a visit, confirm the SOAP draft shows next to original transcript/photos, edit a field, click Approve, confirm status actually locks/updates.
3	Re-test case draft accuracy fix	Re-run the same patient scenario after the S/O/A/P accuracy prompt; confirm Assessment/Plan are now case-specific, not generic.
4	Re-test document upload confidence flagging	Re-upload the handwritten prescription and confirm low-confidence/yellow badges appear; also test a clean "Load sample Rx" for the happy path.
5	Seed realistic demo data	Seed 3 demo patients: one routine, one with an uploaded prescription (for the timeline feature), one that triggers the red-flag path. Add a /kiosk landing page.
6	Rehearse the full demo end-to-end	Time a complete walkthrough (check-in -> intake -> doc upload -> case draft -> doctor approval -> triage flag); record a backup video in case of live Wi-Fi/API issues.
7	Prep talking points	Rehearse the feature breakdown, workflow, tech stack, and the "this already exists" rebuttal table out loud, not just on paper.

Compiled from the full project-planning conversation — problem framing, competitive research, naming, final feature set, workflow, tech stack, 36-hour build plan, Antigravity build prompts, and the live debugging log through pre-demo checklist.